

Decisiones de diseno e implementacion

Proyecto final de Comunicaciones Digitales - modem optico pantalla-camara

1. Lectura de la guia y objetivo del sistema

La guia pide disenar e implementar un modem optico espacio-temporal half-duplex capaz de transmitir un archivo de texto de 500 caracteres desde la pantalla de un computador hacia la camara web de otro computador, a una distancia minima de 50 cm, en 10 segundos o menos y sin conexion de red entre equipos. Tambien exige adaptar conceptos de comunicaciones digitales al canal pantalla-camara: modulacion, sincronizacion, deteccion, codificacion de canal y procesamiento espacial.

El canal descrito por la guia introduce perspectiva, rotacion, escala, cambios de iluminacion, desenfoque, saturacion, ruido del sensor, compresion de video, desacople entre pantalla y camara, y posibles efectos de rolling shutter. Por eso el proyecto se implemento como una cadena completa de comunicaciones con redundancia, calibracion y validacion.

2. Donde queda esto en el proyecto

Este PDF queda en la raiz del repositorio como DECISIONES_IMPLEMENTACION.pdf, junto a los README de sustentacion. El contenido explica decisiones que corresponden a estas partes del codigo:

- common/: configuracion, modulacion, paquetes, ECC, metricas y rendimiento.
- transmitter/: generacion de frames, secuencia multi-frame y visualizacion en pantalla.
- receiver/: deteccion de pantalla, perspectiva, calibracion, demodulacion, recepcion por camara y reconstruccion.
- main_*.py: comandos ejecutables para emisor, receptor, analisis y pruebas por etapas.

3. Resumen de arquitectura final

La arquitectura final se separo en tres capas: codigo comun, transmisor y receptor. La carpeta common contiene el protocolo, modulacion, ECC, metricas y configuracion. La carpeta transmitter convierte mensajes en frames visuales. La carpeta receiver detecta la pantalla, rectifica perspectiva, demodula, reconstruye paquetes y corrige errores.

```
Texto -> bytes -> ECC opcional -> paquetes -> bits -> 00K/4-ASK -> frames PNG -> pantalla  
camara -> deteccion de esquinas -> homografia -> lectura de celdas -> demodulacion -> paquetes -> ECC  
-> texto
```

4. Decisiones principales

4.1 Implementacion incremental por fases

Lo que pide la guia: La guia propone Fase A transmisor, Fase B receptor con deteccion/sincronizacion/decodificacion y Fase C integracion multi-cuadro.

Decision tomada: Se construyo primero una base digital sin camara, luego PNG estatico, foto/camara, perspectiva, ECC, multi-frame, metricas y finalmente 4-ASK.

Por que: Permite validar cada bloque antes de depender del canal real. Si falla la camara, aun se puede comprobar que bits, paquetes, ECC y modulacion funcionan offline.

Como se implemento: Se crearon scripts por etapa y pruebas automaticas. El flujo final se ejecuta con main_tx_sequence.py y main_rx_video_sequence.py.

Archivos clave: main_part1.py, main_tx_static.py, main_rx_offline.py, main_rx_photo.py, main_tx_sequence.py, main_rx_video_sequence.py, tests/

4.2 Grilla visual de macropíxeles

Lo que pide la guía: La guía pide dividir la pantalla en una matriz $M \times N$ de celdas considerando resolución, distancia y desenfoque.

Decision tomada: Se usó una imagen de 1280x720 dividida en 32x18 celdas. Cada celda mide 40x40 píxeles en el frame generado.

Por que: Una celda grande facilita que la cámara promedie el brillo y reduce errores por desenfoque o pequeñas desviaciones. 32x18 equilibra robustez y capacidad.

Como se implementó: FrameConfig centraliza la configuración. Transmisor y receptor usan la misma grilla para dibujar y leer celdas.

Archivos clave: common/frame_config.py, common/frame_layout.py, transmitter/generator.py, receiver/decoder.py

4.3 Marcadores y perspectiva

Lo que pide la guía: La guía pide marcadores fiduciales para detectar pantalla y corregir perspectiva automáticamente.

Decision tomada: Se agregaron marcadores en las cuatro esquinas; en vivo se colorean para facilitar detección.

Por que: La cámara puede ver la pantalla inclinada. La homografía convierte esa vista en una imagen canónica antes de leer la grilla.

Como se implementó: El receptor detecta esquinas y aplica `cv2.getPerspectiveTransform + cv2.warpPerspective`.

Archivos clave: transmitter/generator.py, transmitter/sequence.py, receiver/perspective.py, receiver/sequence_decoder.py

4.4 Pilotos de calibración

Lo que pide la guía: La guía pide celdas de referencia conocidas para calibrar brillo/color y compensar iluminación/exposición.

Decision tomada: Se reservaron 8 celdas piloto: en OOK alternan blanco/negro; en 4-ASK cubren cuatro niveles de gris.

Por que: La cámara no mide niveles ideales. Los pilotos permiten adaptar umbrales según la imagen real.

Como se implementó: El receptor estima umbral OOK y centros de decisión 4-ASK. Si la calibración no es válida, rechaza el frame.

Archivos clave: common/frame_layout.py, receiver/calibration.py, receiver/decoder.py

4.5 OOK y 4-ASK

Lo que pide la guía: La Fase A pide al menos dos esquemas de modulación, incluyendo una opción multinivel.

Decision tomada: Se implementó OOK como modo robusto y 4-ASK como modo de mayor densidad. Cada transmisión usa una sola modulación.

Por que: OOK es más tolerante. 4-ASK lleva 2 bits por celda y reduce tiempo, apoyando el bonus de mayor densidad/velocidad.

Como se implementó: common/modulation.py define niveles, mapeo Gray, bits por símbolo e IDs. Los CLI aceptan `--modulation {ook,4ask}`.

Archivos clave: common/modulation.py, common/packet.py, transmitter/generator.py, receiver/decoder.py, main_tx_sequence.py, main_rx_video_sequence.py

4.6 Protocolo multi-frame

Lo que pide la guía: La Fase C pide estructura de cuadros con sincronización, número de secuencia y delimitador de fin.

Decision tomada: Se diseñó un paquete con magic OM, secuencia, total, longitud, bandera de fin y modulación.

Por que: Un frame no alcanza para 500 caracteres con redundancia. La secuencia permite reconstruir mensajes largos y ordenar paquetes.

Como se implemento: `split_payload` divide el mensaje, `encode_packet` crea cabecera y `decode_packet` valida antes de aceptar.

Archivos clave: `common/packet.py`, `transmitter/sequence.py`, `receiver/sequence_decoder.py`

4.7 Reed-Solomon ECC

Lo que pide la guía: La guía pide implementar al menos una técnica de control de errores.

Decision tomada: Se eligió Reed-Solomon configurable con `--ecc`.

Por que: Es práctico para errores residuales a nivel de bytes y permite recuperar información sin retransmisión explícita.

Como se implemento: El transmisor agrega paridad; el receptor concatena payloads, corrige y reporta `corrected_symbols`. El `--ecc` debe coincidir.

Archivos clave: `common/ecc.py`, `transmitter/sequence.py`, `receiver/sequence_decoder.py`, `main_tx_sequence.py`, `main_rx_video_sequence.py`

4.8 Recepción continua, repetición y votación

Lo que pide la guía: La Fase C pide captura continua y manejo del desacople entre pantalla y cámara.

Decision tomada: El transmisor repite la secuencia y el receptor guarda votos por paquete.

Por que: La cámara puede saltar frames o capturar transiciones. Repetir y votar aumenta la probabilidad de una copia válida.

Como se implemento: `decode_video_stream` captura continuamente; `packet_votes` guarda versiones; los reintentos se acotan para evitar lag.

Archivos clave: `transmitter/sequence.py`, `receiver/sequence_decoder.py`

4.9 BER, tiempo y throughput

Lo que pide la guía: La rubrica evalúa rendimiento: tiempo, BER, distancia, automatización y robustez.

Decision tomada: Se agregaron métricas de BER, frames, tiempo estimado, throughput y muestras de cámara por frame.

Por que: Permite defender el resultado con datos, no solo con una demostración visual.

Como se implemento: `common/metrics.py` calcula BER; `common/performance.py` estima rendimiento; los CLI aceptan mensajes esperados.

Archivos clave: `common/metrics.py`, `common/performance.py`, `main_analyze_performance.py`, `main_rx_sequence_offline.py`, `main_rx_video_sequence.py`

5. Relacion con la rubrica de la guia

Item de la guia	Estado	Evidencia
50 caracteres en cuadro estatico, BER < 1e-2, 30 cm	Cumplido	Frame estatico, receptor offline/foto y pruebas por etapas.
500 caracteres en tiempo real, <= 10 s	Implementado; demostrar en vivo	main_tx_sequence.py + main_rx_video_sequence.py; 4-ASK reduce frames.
BER < 1e-4	Medible; idealmente BER 0	--expected-message-file y common/metrics.py.
Distancia >= 50 cm	Depende de demo presencial	Debe medirse fisicamente.
Deteccion automatica y sincronizacion confiable	Parcial/fuerte	Esquinas, homografia, paquetes con secuencia y repeticion.
Robustez con iluminacion variable y angulo <= 15 grados	Implementado parcialmente; validar en aula	Pilotos, calibracion, perspectiva, ECC.
Bonus: mayor densidad/velocidad < 5 s	Cumplible con 4-ASK	4-ASK, 2 bits/celda, menos frames que OOK.
Bonus: > 2 metros con BER < 1e-4	No demostrado	Requiere modo alcance y prueba fisica.
Bonus: canal dinamico	Reclamable solo si se demuestra moviendo equipos	Rectificacion por frame y votos ayudan, pero debe probarse.
Bonus: full-duplex	No implementado	El sistema actual es half-duplex.

6. Comandos recomendados para demostrar

Demo corta y estable con 4-ASK:

```
python main_rx_video_sequence.py --camera 2 --backend dshow --ecc 16 --modulation 4ask --preview --max-frames 0 --expected-message "Hola mundo"

python main_tx_sequence.py --message "Hola mundo" --ecc 16 --modulation 4ask --show --duration-ms 500 --repeat 20
```

Demo de 500 caracteres:

```
python main_rx_video_sequence.py --camera 2 --backend dshow --ecc 64 --modulation 4ask --preview --max-frames 0 --best-effort-after-seconds 10 --expected-message-file mensaje_500.txt

python main_tx_sequence.py --message-file mensaje_500.txt --ecc 64 --modulation 4ask --show --duration-ms 500 --repeat 30
```

7. Decisiones descartadas o pendientes

- No se implemento full-duplex porque la guia base pide half-duplex y full-duplex es bonus avanzado.
- No se implemento RGB/CSK porque 4-ASK en escala de grises ya cumple la modulacion multinivel con menor dependencia del balance de blancos.
- No se implemento compensacion formal de rolling shutter; se mitiga con duracion de frame, repeticion, votos y ECC.
- El alcance mayor a 2 m no se debe reclamar sin prueba fisica. Para intentarlo: menor grid_cols/grid_rows, OOK, ECC alto, brillo alto y duracion mayor.

8. Mensaje final para sustentacion

La decision central fue construir un modem optico modular y medible. OOK da robustez, 4-ASK da mayor densidad, los marcadores corrigen geometria, los pilotos calibran brillo, Reed-Solomon reduce errores residuales y el protocolo multi-frame permite transmitir mensajes de 500 caracteres. La implementacion sigue las fases de la guia y deja evidencia cuantitativa mediante BER, throughput, tiempo estimado y pruebas automaticas.